

\$ cat portfolio.md

최지환

소프트웨어 엔지니어 / MLOps 엔지니어

jhwan333@gmail.com · linkedin.com/in/jihwan-choi · github.com/jhwanchoi · jhwanchoi.github.io

4.48억

AWS 대비 절감

6개월→1.5개월

검증 시간

99.97%

성공률

8천만/일

로그 이벤트

01 소개

자율주행 AI 개발을 위한 MLOps 추론·평가 플랫폼을 설계하고 운영하는 소프트웨어 엔지니어입니다.

타 엔지니어와 공유 GPU 클러스터(RKE2 Kubernetes, 60+ 이중 GPU)를 공동 설계하며 스케줄러·큐 정책 설계를 주도했고, 추론·평가 워크플로우(SVEvalFlow: PRISM·Inferit·MTBF Validation)를 단독 오너십으로 구축·운영하고 있습니다. 단일 머신으로 6개월 이상 걸리던 20,000시간 규모 검증을 분산·병렬화로 1.5개월에 완료(173,101 clips, 99.97% 성공률)하고, 비용 분석 기반 On-Premise 전환으로 AWS 대비 4.48억원(55%)을 절감했습니다. 비정상 워크로드 분석으로 월 1억원의 과다 정산을 방지하고, 일일 8,000만 건의 이벤트를 처리하는 로그 분석 시스템을 운영하는 등 기술적 해결이 측정 가능한 비즈니스 가치로 이어지는 경험을 쌓아왔습니다.

Validation Team, Data Utilization Team 등 다양한 조직과 협업하며 요구사항을 분석하고, 신규 서비스의 기획부터 아키텍처 설계, 개발, 배포, 운영까지(프로젝트에 따라 설계 또는 전 과정을) 담당합니다. On-Premise와 AWS를 아우르는 하이브리드 인프라 운영 경험을 쌓았고, 복잡한 기술적 문제를 문서화하고 팀과 공유하는 것을 중요하게 생각합니다. 최근에는 LLM Wiki·Multi-Model Debate·Bedrock 비용 관측 등 AI-native 개발 도구를 직접 만들어 사내에 전파하고 있습니다.

02 프로젝트

MLOps 플랫폼 — 공유 GPU 클러스터 공동 설계 & SVEvalFlow 오너

2025.10 — 현재

사내 MLOps 클러스터 공동 설계·구축 및 추론·평가 워크플로우 오너십

MLOps Platform — Shared GitOps + GPU Orchestration

RKE2 cluster · ArgoCD app-of-apps · Airflow / Ray on 4 shared GPU nodes · SVEvalFlow on top

SVEVALFLOW (inference / evaluation)

Inferit · MTBF Validation · PRISM

ORCHESTRATION & MLOPS

Airflow (KubernetesPodOperator) · Ray / KubeRay · MLflow · lakeFS · Harbor · Redis

GITOPS & SECRETS

GitHub Actions (self-hosted ARC) · ArgoCD app-of-apps · External Secrets · Grafana / Prometheus / Loki

RKE2 CLUSTER + STORAGE

control plane · 4 GPU workers (RTX 3090-6000) · service nodes · object store (S3) + NFS

● led scheduler & GPU-quota policy

● KubernetesPodOperator GPU fan-out

● GitOps: GitHub → Harbor → ArgoCD → cluster

● secrets via External Secrets + cloud SM

▶ 배경 및 문제:

- 모델 학습·추론·평가 전 과정의 수동 관리로 인한 비효율
- 실험 추적 및 결과 재현성 부재, 학습/평가 데이터 버전 관리 미흡

- 분산 학습/추론을 위한 이중 GPU 클러스터 통합 관리 체계 필요

▶ 해결 — 공유 클러스터 아키텍처 (타 엔지니어와 공동 설계):

- RKE2 Kubernetes 기반 하이브리드(GPU/CPU) 클러스터 공동 설계·구축; 60+ 이중 GPU(RTX 3090/4080/4090/5090, RTX 6000)
- NGINX Gateway Fabric 기반 서비스 라우팅, Cert-Manager TLS 관리, NFS CSI 공유 스토리지, MetalLB 베어메탈 로드밸런싱
- 스케줄러·큐 정책 설계 주도: nodeSelector + GPU 타입별 tolerations로 이중 GPU 워크로드 분배, Resource Quota·Priority Class로 핵심 Job 우선순위 보장
- 15+ 핵심 컴포넌트 통합: Ray/KubeRay, MLflow, Airflow, ArgoCD, KServe, Harbor, Nexus 등
- 학습 워크플로우(SVDeepFlow)는 협업, 추론·평가 워크플로우(SVEvalFlow)는 단독 오너십

▶ 기술 선택 배경:

- RKE2: EKS 대비 On-Premise GPU 서버 직접 관리에 적합하며 보안 기본값 우수
- Ray/KubeRay: Python-native API로 접근성 확보, RayJob CRD로 Kubernetes 네이티브 배치 학습/추론 지원
- KServe: TorchServe/Triton 직접 운영 대비 Knative 기반 Scale-to-Zero로 GPU 유휴 비용 절감
- ArgoCD: Flux CD 대비 직관적 UI와 멀티 클러스터 지원, Helm/Kustomize 네이티브 지원

▶ 도전과 해결:

- 특정 CPU 노드 풀에서 pip install 시 비정상적 timeout 빈발 → Nexus 프록시 레지스트리 구축으로 종속성 캐싱
- Ray Worker Pod 간 protobuf 버전 충돌로 RayJob 실패 → requirements.txt 버전 pin 및 Docker multi-stage build로 의존성 격리
- 사내 private repository SSH clone 시 token prompt 이슈 → HTTP clone + 환경변수 기반 인증으로 전환

▶ CI/CD 파이프라인 3종:

- Pipeline 1 (Airflow 경유): GitHub → Image Build → Airflow DAG → ArgoCD → RayJob 분산 학습
- Pipeline 2 (ArgoCD 다이렉트): GitHub → Image Build → ArgoCD → RayJob 즉시 실행
- Pipeline 3 (추론 배포): GitHub → ArgoCD → KServe/Knative 서버리스 추론 (Scale-to-Zero)

▶ 모니터링 및 아티팩트 관리:

- Grafana + Prometheus + Loki 통합 모니터링 대시보드
- Harbor 프라이빗 컨테이너/Helm Chart 레지스트리, Nexus 아티팩트 저장소(종속성 캐싱)
- MLflow 실험 추적 및 모델 아티팩트 관리

아키텍처: GitHub → GitHub Actions → Harbor → (Airflow DAG / ArgoCD GitOps / KServe) → Ray / KubeRay (분산 GPU) → MLflow, NFS, Grafana

성과: 60+ 이중 GPU 공유 클러스터 공동 설계 (스케줄러·큐 정책 주도) / 학습·추론·배포 자동화 CI/CD 3종 / 15+ 오픈소스 MLOps 컴포넌트 Kubernetes 통합 배포

기술: Python, Kubernetes (RKE2), Ray, KubeRay, KServe, Knative, MLflow, Airflow, ArgoCD, Harbor, Nexus, GitHub Actions, Helm, Grafana, Prometheus, Loki, NFS CSI, MetalLB

Inferit — svnet3 빌드·추론 가속 & 형상관리

2025 - 현재

svnet3 빌드 형상관리와 분산·병렬 추론, 평가 자동화 (SVEvalFlow 핵심)

Inferit – svnet3 Build & Distributed Inference

SVEvalFlow component · FastAPI BFF · Airflow → KubernetesPodOperator GPU fan-out



▶ 배경 및 문제:

- svnet3(사내 자율주행 비전 모델) SW 버전별 빌드 인자 파편화로 자동화 어려움
- 고객사가 티켓에 첨부하는 option 파일(카메라 파라미터 등)이 체계적으로 관리되지 않아 탐색에 시간 과다 소모
- 단일 머신 추론으로 대규모 검증/재추론에 과도한 시간 소요
- 평가 결과물 수동 Copy & Paste, 버전 추적 불가

▶ 해결 — 빌드 형상·아티팩트 관리:

- svnet3를 다양한 config로 빌드하는 버전/형상/아티팩트 관리 체계 구축
- 내부 자동화용/고객사용 Docker Image 이원화 설계 → 내부 이미지에 일원화된 entrypoint 및 ini 템플릿 포함으로 버전별 실행 인자 파편화 해소

▶ 해결 — 고객 option 파일 형상관리:

- 고객사 option 파일(카메라 파라미터 등)을 버전/형상 관리하여, 엔지니어 ↔ PM 간 1~2일 걸리던 탐색·커뮤니케이션 로드 제거

▶ 해결 — 분산·병렬 추론 가속:

- 단일 머신 추론을 클러스터 GPU로 분산·병렬화 → 노드 내 GPU 수에 비례한 가속
- 극단적 사례: 20,000시간 데이터에 대한 MTBF 검증이 기존 예상 6개월 이상 → 1.5개월로 단축

▶ 해결 — 평가 자동화 (Evaluation Driven Development 통합):

- 평가 결과물(xlsx) 수동 Copy & Paste 문제 → MLflow에 metrics 자동 로깅하여 버전별 성능 비교 자동화
- 평가 데이터(256GB+) 버전 관리 → lakeFS로 Git-like commit/branch 기반 데이터 스냅샷 관리

▶ 기술 선택 배경:

- lakeFS: DVC 대비 Git-like 브랜칭 모델과 S3/HCP 호환 인터페이스 제공, 기존 HCP 스토리지와의 호환성이 핵심 선정 기준
- MLflow: Weights & Biases 대비 오픈소스로 사내 배포 가능, MLOps 클러스터와 통합 운영

▶ 성과:

- 20,000h 검증 예상 6개월+ → 1.5개월 (단일 머신 대비)
- 고객 option 파일 형상관리로 1~2일 커뮤니케이션 제거
- 메이저 릴리즈 5+ 버전 호환, 이슈 티켓 기반 자동 추론

기술: Python, Ray, MLflow, lakeFS, Docker, Kubernetes

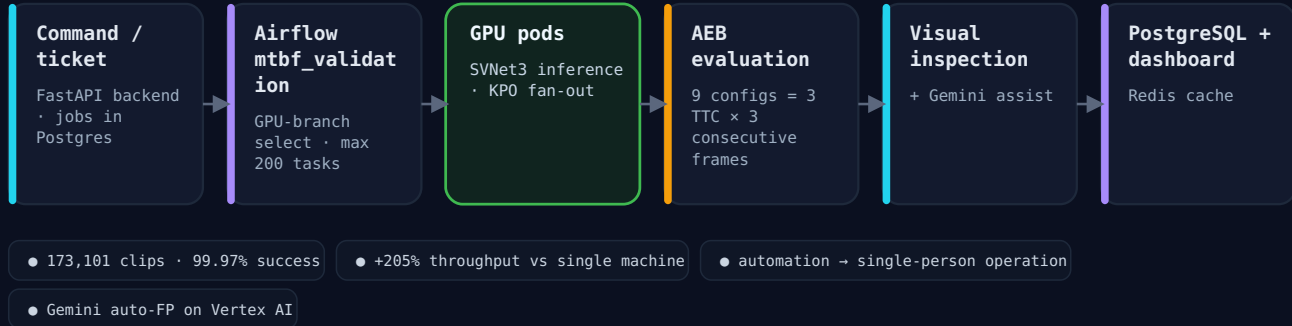
MTBF Validation — svnet3 대규모 신뢰성 검증

2022 – 2025

SVNet3 자율주행 모델의 대규모 신뢰성 검증 자동화 서비스 (SVEvalFlow)

MTBF Validation – Large-Scale Reliability Pipeline

SVEvalFlow component · Airflow fan-out · AEB evaluation · Gemini-assisted inspection



▶ 배경 및 문제:

- 20,000시간 주행 영상 데이터(PB급)에 대한 MTBF(Mean Time Between Failures) 검증 필요
- 다양한 해상도(2MP, 3MP, 8MP)와 영상 길이(30초~5분) 혼재로 정확한 리소스 추정 어려움
- 한정된 컴퓨팅 리소스와 예산, 단기 처리 완료 요구
- 기존엔 규모의 한계로 소규모 검증만 가능했으나, 대규모 검증 요구를 더 이상 미룰 수 없게 됨

▶ 해결 — 분산 처리 아키텍처 설계:

- Inferit의 분산·병렬 추론을 활용해 검증 가속
- Airflow DAG로 Build → Inference → KPI 파이프라인 자동화
- 7종 데이터 검증 자동화 파이프라인 (FormatStatus, FrameDrop, ValueRange 등)
- 실시간 모니터링 대시보드 + Evaluation & Visual Inspection 통합 UI (이슈 프레임 전후 영상 확인, 통계 제공)
- Gemini 보조 검토 도입: 반복적 확인 부담 완화, 단 최종 판단은 사람이 수행

▶ 기술 선택 배경:

- Airflow: Celery/Luigi 대비 DAG 기반 시각적 파이프라인 관리, 웹 UI로 비개발자(Validation Team)도 진행 상황 모니터링 가능
- Kubernetes 분산 처리: Slurm 대비 동적 스케줄링과 리소스 격리(namespace/RBAC) 우수, 이미 사내 K8s 인프라 존재

▶ 도전과 해결:

- 이종 GPU 혼합 클러스터에서 Job 스케줄링 불균형 → nodeSelector + GPU 타입별 tolerations 설정으로 워크로드 분배 최적화
- On-Premise NFS ↔ S3 간 대규모 데이터 전송 병목 → 비용 분석을 통한 하이브리드 스토리지 전략 수립
- Airflow DAG 실행 시 GPU Pod 리소스 경합 → Resource Quota + Priority Class 설정으로 검증 Job 우선순위 보장

▶ 인프라 전환 (Cloud → On-Premise):

- 비용 분석(AWS 8.2억 vs On-Premise 3.7억, 55% 절감) 기반 On-Premise 전환 제안 및 실행
- RTX 3090/4080/4090 등 GPU 장비 도입을 위한 벤치마크 실험 및 사양 제안 (하드웨어 구매 의사결정 지원)
- HCP Object Storage 활용 (S3 대비 일회성 비용으로 월과금 제거)

▶ GPU 클러스터 운영:

- 60+ GPU 리소스를 Rancher 기반 K8s 클러스터로 관리
- CUDA 설정, GPU job 테스트, 장애 대응 트러블슈팅

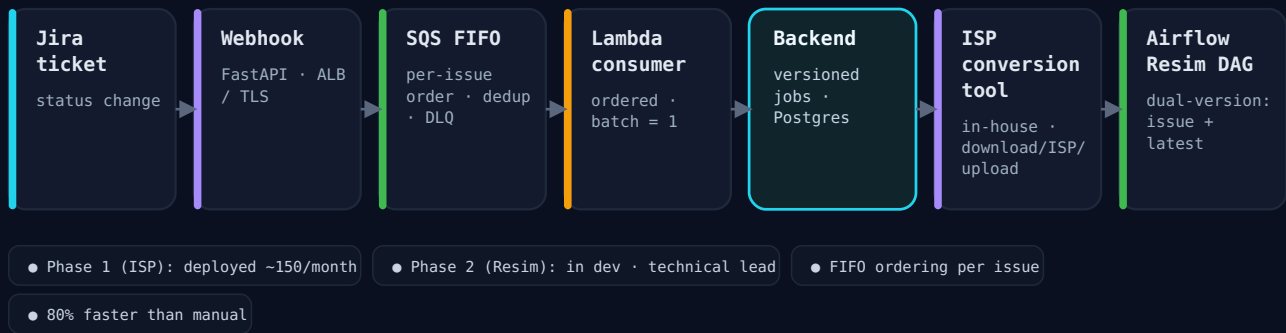
아키텍처: Git Repo (svnet3) → Airflow 스케줄러 → K8s Pods (GPU ×60+) → (Build / Inference / KPI) → PostgreSQL / HCP (결과)

성과: 173,101 clips 처리(99.97% 성공률) / 처리 수량 205% 증가 / 처리 시간 67.3% 감소(예상 6개월+ → 1.5개월) / AWS 8.2억 → On-Premise 3.7억(55% 절감) / 자동화로 대규모 검증을 1인 운영

기술: Python, FastAPI, Airflow, Kubernetes, PostgreSQL, HCP Object Storage, Rancher

PRISM – Jira-Driven ISP Conversion & Re-Simulation

event-driven · FIFO-ordered per issue · Terraform IaC



▷ 배경 및 문제:

- 고객 Jira 티켓 기반 ISP 변환 요청 수동 처리
- 변환 상태 추적 어려움; Re-simulation 작업과의 연계 부재

▷ 해결 — 이벤트 기반 아키텍처:

- Jira Webhook으로 티켓 이벤트 수신; AWS SQS + Lambda로 비동기 처리
- 상태 변경 시 Slack 알림 통합, 처리 이력 대시보드 제공

▷ 해결 — Infrastructure as Code & 연동:

- Terraform으로 VPC, EC2, SQS, Lambda 프로비저닝, dev/prod 환경 분리
- 사내 ISP 변환툴 연동, Callback 기반 상태 업데이트; Inferit에 추론 요청, 결과 이력 관리

▷ Re-simulation 파이프라인 (Phase 2, 개발 중):

- 초기 개발 후 **테크니컬 리드**로 타 조직 개발자의 개발 현황 파악·관리 (비개발 조직 대상 지원)
- ISP 변환 완료 후 Airflow DAG 트리거로 svnet3 Re-simulation 자동 실행
- SWIT Docker Image 기반 이슈 버전/최신 버전 동시 추론
- 처리 범위: CONVERSION_ONLY / FULL (Download → ISP → Upload → Resim) / RESIM_ONLY
- PRISM Dashboard 수동 트리거, Resim 결과 Jira 댓글 자동 작성

▷ 기술 선택 배경:

- 이벤트 기반(Jira Webhook → SQS → Lambda): ISP 변환 요청이 비동기적·불규칙하여 서버리스가 비용 효율적. ECS 상시 운영 대비 Lambda로 idle 비용 제로
- Terraform: CloudFormation 대비 멀티 클라우드 지원 및 HCL 가독성 우수, workspace로 dev/prod 분리

▷ 도전과 해결:

- Jira Webhook 이벤트 중복 수신 → SQS FIFO Queue의 deduplication ID로 idempotency 보장
- 변환 결과의 Jira 자동 피드백을 위한 Callback 패턴 설계 → 변환 완료 시 상태 업데이트, 실패 시 재시도 + 3회 실패 시 담당자 Slack 알림

아키텍처: Jira 티켓 → Webhook → AWS SQS → Lambda → PRISM BE (FastAPI) → (PostgreSQL / 사내 ISP 변환툴 / Slack)

성과: 월 평균 ~150건 ISP 변환 요청 자동화(배포·운영 중) / 수동 대비 80% 단축 / Terraform 기반 IaC로 환경 프로비저닝 자동화

기술: Python, FastAPI, PostgreSQL, AWS (SQS, Lambda), Terraform, Jira Webhook, Airflow

WLS — Workload Logging Service

2023 — 2025

라벨링 작업 로그 분석 및 외주 정산 자동화 시스템

▷ 배경 및 문제:

- 라벨링 작업자의 워크로드 분석 체계 부재; 외주 정산 시 수동 검증으로 인한 오류; 비정상 워크로드로 인한 과다 비용

▶ 해결 — 대규모 로그 처리·분석:

- AWS SQS 기반 로그 수집, MongoDB로 일일 8,000만 건 이벤트 저장 및 분석; MongoDB Atlas 마이그레이션으로 안정성 확보
- State Machine 기반 워크로드 분석: 30+ 상태 정의로 라벨링 패턴 분석, 비정상 워크로드 자동 탐지
- 정산 자동화: aggregation pipeline으로 정산 데이터 자동 산출, 신규 feature(OD, SOD, TSTLD, RMD) 정산 지원
- WLS 로그 분석으로 작업 병목 규명 → UI/UX 개선 및 ALAS 자동 라벨링 모델로 연결, 라벨링 효율 20% 향상

▶ 기술 선택 배경:

- MongoDB: PostgreSQL 대비 스키마리스 구조로 30+ 상태의 비정형 로그에 적합, 일일 8,000만 건 write-heavy 워크로드에서 document-level locking 유리
- AWS SQS: Kafka 대비 관리 오버헤드 최소화, 로그 수집 특성에 부합

▶ 도전과 해결:

- MongoDB 단일 인스턴스에서 80M/day 처리 시 write 성능 저하 → Atlas 마이그레이션으로 sharding/replica set 구성, 백업 주기 최적화(12주 → 20주)로 스토리지 비용 절감
- 비정상 워크로드 패턴(동일 객체 반복 수정, 비활성 상태 시간 기록 등) 탐지 → 30+ 상태(Idle, Active, Paused, Reviewing, Submitted, AFK 등) 전이 규칙 기반 State Machine으로 분류
- 외주 업체별·Feature별 비용 산출 복잡도 → aggregation pipeline의 \$group, \$unwind, \$lookup 체이닝으로 자동 집계

아키텍처: Labeling Tool → AWS SQS → WLS BE (Django) → MongoDB (일 8천만 건) → (State Machine / Aggregation / Billing)

성과: 과다 정산 방지 월 약 1억원(정산 감사 확정치, 2024 상반기 누적 약 6억원) / 일일 8,000만 건 이벤트 처리 / 30+ State Machine / 라벨링 효율성 20% 향상

기술: Python, Django, MongoDB, AWS SQS, EC2, MongoDB Atlas

ALAS — Auto Labeling Assistant Service

2024

객체 자동 생성 모델 운영 서비스

▶ 문제 & 해결:

- 라벨링 작업자의 워크로드 과다(WLS 로그 분석으로 병목 규명); 다양한 모델 지원 필요
- FastAPI 기반 마이크로서비스, AWS SNS/SQS 메시징; AWS ECS → EKS 마이그레이션 설계, CDK 기반 IaC
- 모델 지원: Ground Fitting OD/FSD, Occlusion Score

▶ 기술 선택 & 도전:

- FastAPI: Flask 대비 비동기(async/await) 네이티브 지원으로 추론 I/O 대기 최적화, Pydantic 기반 자동 API 문서화
- SNS/SQS 팬아웃: 단일 라벨링 요청에 여러 모델이 동시 추론(SNS → 복수 SQS)
- 다중 모델 응답 시간 편차 → 비동기 팬아웃 후 병합, 가장 느린 모델 기준 타임아웃
- CDK 기반 ECS 스택을 EKS 전환 시 Helm Chart로 변환, CDK는 인프라 계층(VPC, SQS 등)에만 유지

▶ 성과:

- 객체 자동 생성으로 수동 라벨링 워크로드 약 35% 감소; 다중 모델 운영; ECS → EKS 마이그레이션으로 운영 효율화

기술: Python, FastAPI, AWS ECS/EKS, SNS/SQS, CDK

- ▶ **IGS (Ingestion Service, 2024):** 외부 고객사 데이터 수집·변환 서비스 기획 및 아키텍처 설계; REST API/DB 스키마 설계, Airflow 기반 변환 파이프라인 아키텍처 정의 (설계 전담).
- ▶ **SURF Compass Service (2025):** 데이터 수집 장비 모니터링 서비스 기획 및 설계(설계 전담); FastAPI 기반 REST API/DB 설계.
- ▶ **RNS (Release Note Service, 2023):** NestJS + Atlassian API로 Jira 티켓 기반 릴리즈 노트 자동 생성; 작성 시간 50% 단축 (4시간 → 2시간).
- ▶ **SLT (Smart Labeling Tool, 2022):** C++/MFC로 2D LD 라벨링 툴 개발; 생산성 30% 향상.

03 기술 스택 요약

Backend	Python · Django · FastAPI · NestJS
Database	PostgreSQL · MongoDB · Redis
Cloud	AWS (EC2, EKS, ECS, SQS, SNS, Lambda, S3, Bedrock)
Infrastructure	Kubernetes (RKE2) · Docker · Terraform · CDK · Rancher
Workflow	Airflow · Celery
Monitoring	Grafana · Prometheus · Loki
ML / MLOps	Ray · KubeRay · MLflow · KServe · Knative · lakeFS
CI / CD	ArgoCD · GitHub Actions · Harbor · Helm
AI / LLM	Claude Code · Gemini CLI · Codex · MCP · AWS Bedrock · n8n

LLM Wiki — MLOps 운영 지식 관리 자동화

2025 — 현재

RAG Knowledge Wiki — Hybrid Retrieval + Grounded Synthesis

CI-rebuilt embedding index · hybrid keyword + vector retrieval · cited answers



- ▶ Confluence/외부 의존성을 제거하고 Git + Markdown 기반 LLM Wiki 구조로 재설계.
- ▶ 주요 기능: 자연어 기반 문서 작성(frontmatter.commit/push 자동화), 운영 문서 검색, 관련 문서 기반 질의응답, 품질 린트(stale/schema/orphan/broken-link 검사), 수동 동기화.
- ▶ 데이터 신선도 전략: 기본 local grep 검색 + 일정 시간 경과 시 fresh sync 유도 + write 시 pull → commit → push.
- ▶ 목적: 개인에게 흩어진 MLOps 운영 지식을 팀 단위 자산으로 축적, 사용 마찰 제거.

클러스터 운영 Claude 플러그인

2025

- ▶ 진단·스토리지 등 MLOps 클러스터 운영 유틸리티를 슬래시 커맨드로 제공.
- ▶ 문서 기능과 운영 유틸을 역할별로 분리하여 유지보수성 향상.

Multi-Model Debate Hooks

2025

Multi-Model Debate — Self-Critique Across Models

@debate hook · cross-model review via AWS Bedrock + Gemini CLI · feedback loop to Claude



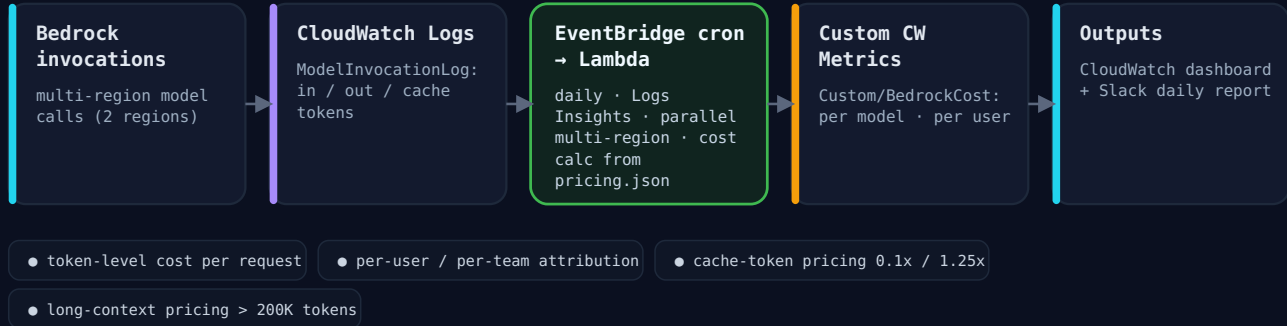
- ▶ 단일 모델의 환각·편향을 줄이기 위해 여러 LLM이 서로 검토하는 워크플로우 설계.
- ▶ @debate: Claude 초안 → GPT 비판적 분석 → Gemini 제3자 리뷰 → Claude 종합.
- ▶ 설계 검토·중요 문서 리뷰 등 품질이 중요한 작업에 적용 (latency/비용 증가는 트레이드오프로 인지).

AWS Bedrock 사용량/비용 관측

2025

AWS Bedrock – LLM Cost & Usage Observability

daily multi-region batch · token-level cost attribution · Terraform IaC



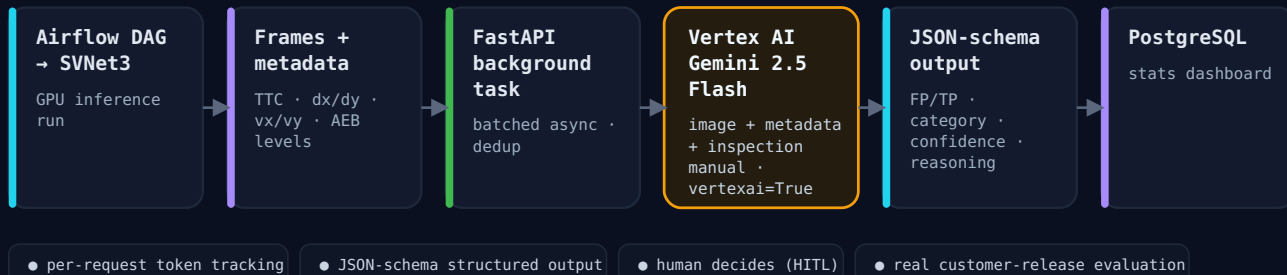
- ▷ 사내 LLM 사용량 증가에 따른 비용 가시성 확보를 위한 모니터링 인프라 구축.
- ▷ 토큰 사용량 및 비용 추정, CloudWatch 대시보드 및 Slack 알림 기반 운영.
- ▷ AI 도구 도입에서 끝내지 않고 비용·사용량을 관측 가능한 운영 체계로 관리.

MTBF Visual Inspection AI 보조

2025

Vertex AI × Gemini – Auto False-Positive Detection

production MTBF feature · structured FP/TP classification · human-in-the-loop



- ▷ 대규모 추론 결과 검증에서 사람이 모든 결과를 직접 확인해야 하는 부담을 Gemini로 완화.
- ▷ Gemini를 최종 판정자로 쓰지 않고, 우선 확인할 케이스 선별·검토 보조로 활용 (최종 판단은 사람).

업무 자동화 프로토타입 (n8n)

2025

- ▷ Slack, GitHub, 모니터링 시스템, 외부 API를 연결한 AI 업무 자동화 프로토타이핑.
- ▷ PR 요약·1차 리뷰 보조, 기술 뉴스봇, 클러스터 모니터링 초동 대응 보조, 채용 보조(이력서 요약·면접 질문 초안 — 사람 검토 용).

사내 전파

2025 - 현재

- ▶ AI-driven development 사내 미팅 호스트, 최신 AI 개발 도구(Claude Code·Gemini CLI·Codex, Plugin/MCP/Skill) 트렌드 전파.
- ▶ AI-native 개발 문화 확산에 기여.

05 연락처

이메일: jhwan333@gmail.com
choi-717554233/

GitHub: github.com/jhwanchoi

LinkedIn: [linkedin.com/in/jihwan-](https://linkedin.com/in/jihwan-choi-717554233/)